UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ – UTFPR

Campus Medianeira

Erik Mazzuco
Letícia Moro
Leonardo Herrero

ÁRVORES ESPACIAIS E GEOMÉTRICAS

https://github.com/leohsm/vp-tree

VP-Tree (Vantage-Point Tree)

Medianeira

2026

Contextualização Histórica e Motivação

Grande parte dos problemas modernos de computação envolve, de alguma forma, responder à pergunta "quais objetos da minha coleção são mais parecidos com este?". Essa é a chamada **busca por similaridade** ou **busca por proximidade**, presente em mecanismos de busca por imagens, reconhecimento facial, correção ortográfica, bioinformática, sistemas de recomendação e classificadores do tipo *k-vizinhos mais próximos* (k-NN). Em todos esses casos, a resposta ingênua consiste em comparar a consulta com todos os *n* elementos da base, o que custa O(n) cálculos de distância por consulta, um custo proibitivo quando a base possui milhões de objetos e a função de distância é cara (por exemplo, comparar duas imagens ou duas sequências de DNA).

As primeiras estruturas de indexação espacial, como a **KD-Tree** (BENTLEY, 1975), resolvem bem esse problema quando os dados são pontos em um espaço vetorial de baixa dimensão. A KD-Tree divide o espaço com hiperplanos alinhados aos eixos, escolhendo a cada nível uma coordenada de corte. Essa abordagem, porém, possui duas limitações importantes. A primeira é que ela exige que os objetos tenham **coordenadas**: não há como aplicar uma KD-Tree diretamente a um conjunto de palavras comparadas pela distância de edição, ou a um conjunto de sequências genéticas. A segunda é a chamada **maldição da dimensionalidade**: à medida que o número de dimensões cresce, os cortes alinhados aos eixos deixam de separar bem os dados e a busca acaba visitando quase toda a árvore.

Para contornar essas limitações, surgiram as estruturas de **indexação métrica**, que não dependem de coordenadas, mas apenas de uma função de distância *d* que satisfaça os axiomas de um **espaço métrico**: não negatividade, identidade ($d(x, y) = 0$ se e somente se $x = y$), simetria ($d(x, y) = d(y, x)$) e, principalmente, a **desigualdade triangular**: $d(x, z) \leq d(x, y) + d(y, z)$. É essa última propriedade que permite descartar regiões inteiras do espaço sem calcular a distância até cada elemento delas.

Em 1991, Jeffrey Uhlmann descreveu as chamadas *metric trees*, árvores que particionam um espaço métrico usando bolas centradas em elementos da própria base (UHLMANN, 1991). Em 1993, Peter Yianilos, de forma independente, formalizou e analisou essa ideia sob o nome de **Vantage-Point Tree (VP-Tree)**, ou "árvore de pontos de vantagem" (YIANILOS, 1993). Yianilos mostrou como construir a árvore de forma balanceada usando a mediana das distâncias, propôs heurísticas para escolher bons pontos de vantagem e apresentou algoritmos de busca do vizinho mais próximo com tempo esperado logarítmico para dados de baixa dimensão intrínseca.

A ideia central da VP-Tree é intuitiva: em vez de perguntar "de que lado de um plano o ponto está?", como na KD-Tree, ela pergunta "o ponto está **perto** ou **longe** deste ponto de referência?". Cada nó escolhe um elemento da base, o *vantage point*, e um raio μ. Os elementos mais próximos que μ vão para uma subárvore e os demais vão para a outra. Como só são necessárias distâncias, a mesma estrutura funciona para pontos no plano com distância euclidiana, para palavras com distância de Levenshtein, para vetores binários com distância de Hamming ou para coordenadas geográficas com a distância sobre a superfície da Terra.

Assim, a VP-Tree destaca-se como uma das estruturas mais simples e elegantes para busca em espaços métricos gerais. Com apenas um ponto e um número real armazenados em cada nó, ela permite responder consultas de vizinho mais próximo e de busca por faixa visitando, em média, apenas uma pequena fração dos nós da árvore.

Estruturação da Árvore

A VP-Tree é uma **árvore binária** em que cada nó armazena um elemento da base de dados, chamado de **ponto de vantagem** (*vantage point*, vp), e um valor real **μ** (mu), que representa um raio de corte. Diferentemente de uma árvore binária de busca, a ordem entre os elementos não é definida por comparação de chaves, mas pela **distância** em relação ao ponto de vantagem de cada nó.

A estrutura da VP-Tree é definida por uma única propriedade fundamental, chamada aqui de **propriedade de partição por bola**. Para todo nó N com ponto de vantagem vp e raio μ:

1. **Subárvore interna (esquerda):** contém apenas elementos x tais que $d(vp, x) < \mu$, ou seja, elementos que estão **dentro** da bola de centro vp e raio μ.
2. **Subárvore externa (direita):** contém apenas elementos x tais que $d(vp, x) \geq \mu$, ou seja, elementos que estão **sobre ou fora** dessa bola.

Na construção balanceada, o raio μ é escolhido como a **mediana** das distâncias entre o ponto de vantagem e os demais elementos do seu conjunto. Dessa forma, cada subárvore recebe aproximadamente metade dos elementos e a altura da árvore fica em torno de $\log_2 n$. Os nós folha guardam apenas um ponto e não possuem raio significativo ($\mu = 0$).

A representação em C de um nó é bastante enxuta: o ponto armazenado, o raio e dois ponteiros. Nenhuma informação de balanceamento (altura, cor, fator de balanceamento) precisa ser mantida:

```
typedef struct VPNode {
    Ponto ponto;              // ponto de vantagem (vantage point)
    double mu;                // raio de corte (mediana das distâncias)
    struct VPNode *interno;   // elementos com d(ponto, x) <  mu
    struct VPNode *externo;   // elementos com d(ponto, x) >= mu
} VPNode;
```

É importante observar que, geometricamente, cada nó divide a região do espaço que recebeu em duas partes: uma **bola** e o seu **complemento** (uma "casca" ao redor da bola). Ao descer na árvore, as regiões vão sendo recortadas por novas bolas, centradas em outros pontos. Essa decomposição é chamada de *ball partitioning* e é ilustrada na Figura 1.

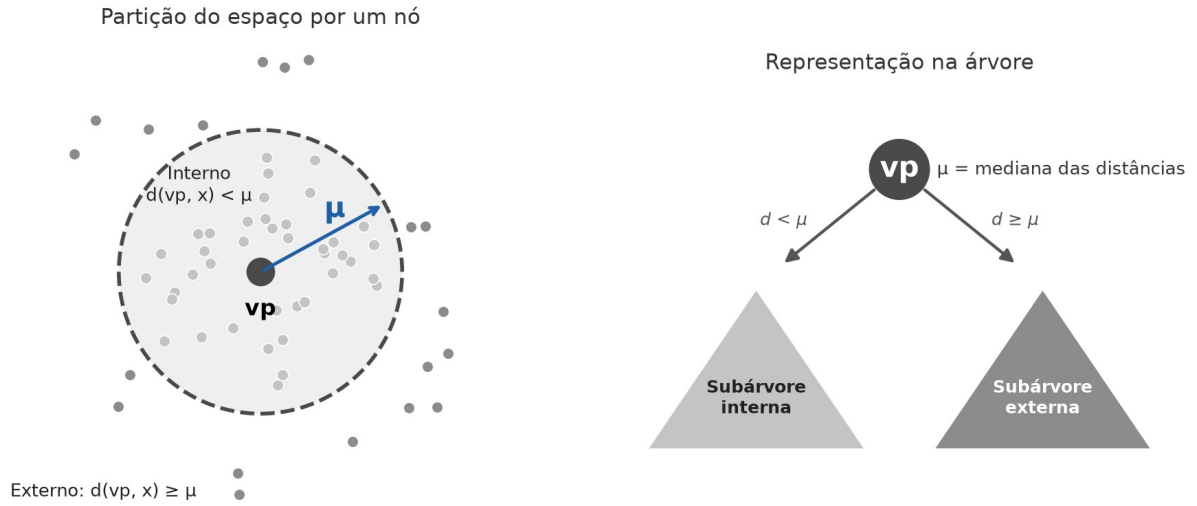


Figura 1

Figura 1: Estrutura de um nó da VP-Tree. À esquerda, o ponto de vantagem (vp) e a bola de raio μ dividem o conjunto em elementos internos (cinza claro, d(vp, x) < μ) e externos (cinza escuro, d(vp, x) ≥ μ). À direita, a representação correspondente na árvore: a subárvore interna fica à esquerda e a externa à direita. Como μ é a mediana das distâncias, as duas subárvores possuem aproximadamente o mesmo número de elementos.

Mecanismos de Operação: A Construção (BUILD)

Assim como a operação de fusão é o coração de uma Skew Heap, na VP-Tree a operação fundamental é a **construção** a partir de um conjunto de elementos. A VP-Tree foi concebida originalmente como uma estrutura **estática**: dado o conjunto completo de dados, ela é construída de uma só vez, de forma balanceada. Como será visto adiante, a remoção também se apoia na construção, reconstruindo apenas a subárvore afetada.

Construção

A construção de uma VP-Tree a partir de um conjunto S de elementos ocorre recursivamente seguindo estes passos:

1. Se S for vazio, retorna-se uma árvore vazia (NULO).
2. Escolhe-se um elemento de S para ser o **ponto de vantagem** do nó e retira-se esse elemento de S.
3. Calcula-se a distância entre o ponto de vantagem e cada um dos elementos restantes.
4. Define-se μ como a **mediana** dessas distâncias (para uma quantidade par de valores, a média dos dois valores centrais).
5. Divide-se S em S_in = { x : d(vp, x) < μ } e S_out = { x : d(vp, x) ≥ μ }.
6. Constrói-se recursivamente a subárvore interna com S_in e a subárvore externa com S_out.

A escolha do ponto de vantagem influencia a qualidade da árvore. A forma mais simples é escolhê-lo **aleatoriamente**, o que já produz bons resultados na prática. Yianilos (1993) propõe uma heurística melhor: sortear alguns candidatos e, para cada um, medir a **dispersão** (variância) das distâncias até uma amostra de elementos, escolhendo o candidato de maior dispersão. Pontos com distâncias bem

espalhadas, em geral localizados nas “bordas” do conjunto, produzem cortes mais nítidos e, consequentemente, mais podas durante a busca.

PSEUDOCÓDIGO DA OPERAÇÃO DE CONSTRUÇÃO

```
Função Construir(Lista S):
    Se S for vazia:
        Retorne NULO

    Nó N = criar_no()
    N.ponto = Escolher_Vantage_Point(S)    // aleatório ou heurística
    Remover N.ponto de S

    Se S for vazia:                        // N é uma folha
        N.mu = 0
        Retorne N

    // Distâncias do ponto de vantagem a todos os demais
    Para cada x em S:
        dist[x] = d(N.ponto, x)

    N.mu = Mediana(dist)

    S_in  = { x em S | dist[x] <  N.mu }
    S_out = { x em S | dist[x] >= N.mu }

    N.interno = Construir(S_in)
    N.externo = Construir(S_out)
    Retorne N
```

Exemplo de construção

Considere o conjunto de sete pontos no plano A(5,5), B(6,6), C(3,4), D(5,8), E(9,2), F(1,9) e G(10,7), com a distância euclidiana. Para fins didáticos, o ponto de vantagem de cada nó será sempre o **primeiro** elemento da lista recebida (em uma implementação real, ele seria sorteado). O processo decorre da seguinte forma:

1. **Raiz (A):** calculam-se as distâncias de A aos demais pontos:
   - d(A,B) = 1,41; d(A,C) = 2,24; d(A,D) = 3,00; d(A,E) = 5,00; d(A,G) = 5,39; d(A,F) = 5,66.
   - A mediana das seis distâncias é (3,00 + 5,00) / 2 = **4,00**, logo μ(A) = 4,00.
   - Subárvore interna (d < 4,00): {B, C, D}. Subárvore externa (d ≥ 4,00): {E, F, G}.
2. **Subárvore interna (B):** com o conjunto {B, C, D}, o ponto de vantagem é B:
   - d(B,C) = 3,61 e d(B,D) = 2,24; a mediana é (2,24 + 3,61) / 2 = **2,92**.
   - D (2,24 < 2,92) vai para a subárvore interna de B; C (3,61 ≥ 2,92) vai para a externa.
3. **Subárvore externa (E):** com o conjunto {E, F, G}, o ponto de vantagem é E:
   - d(E,G) = 5,10 e d(E,F) = 10,63; a mediana é (5,10 + 10,63) / 2 = **7,86**.
   - G vai para a subárvore interna de E e F para a externa.
4. **Folhas:** D, C, G e F ficam sozinhos em seus conjuntos e se tornam folhas (μ = 0).

O resultado é uma árvore perfeitamente balanceada, de altura 3, mostrada na Figura 2. Note como as bolas de raio μ, desenhadas no plano, separam exatamente os pontos de cada subárvore.

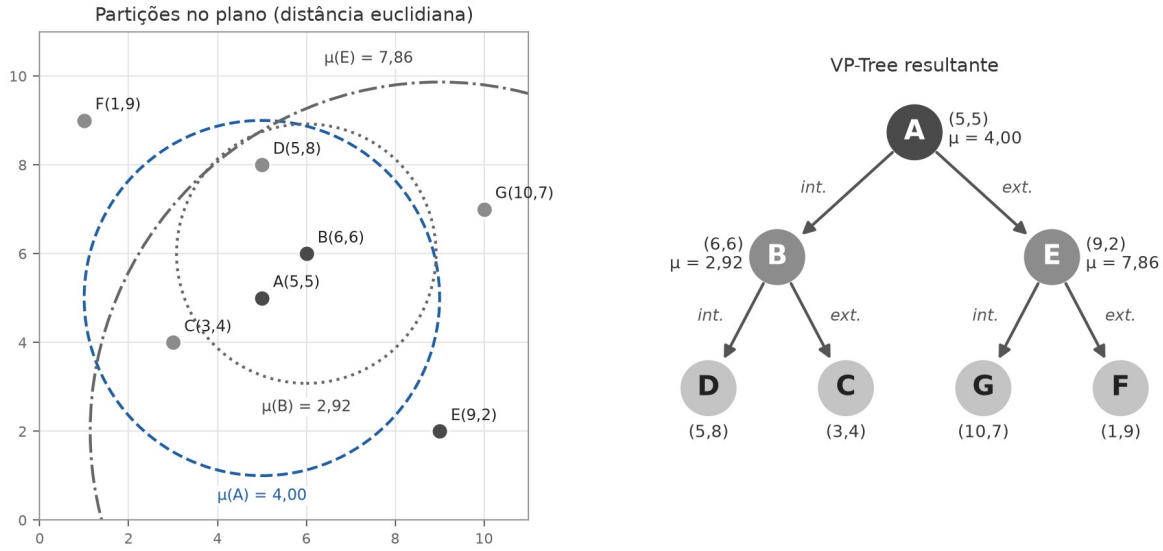


Figura 2

Figura 2: Construção da VP-Tree do exemplo. À esquerda, os sete pontos no plano e as bolas de corte de cada nó interno: a bola de A (μ = 4,00, tracejada) separa {B, C, D} de {E, F, G}; a bola de B (μ = 2,92, pontilhada) separa D de C; e a bola de E (μ = 7,86, traço-ponto) separa G de F. À direita, a árvore resultante, em que cada aresta "int." leva à subárvore interna e cada aresta "ext." leva à subárvore externa.

Inserção em uma VP-Tree

A inserção de um novo elemento x em uma VP-Tree já construída é feita por uma **descida** semelhante à de uma árvore binária de busca, mas guiada por distâncias. Em cada nó, calcula-se d(vp, x) e compara-se com μ: se a distância for menor que μ, a descida continua pela subárvore interna; caso contrário, pela subárvore externa. Ao alcançar uma posição vazia, o novo elemento é colocado ali como uma folha.

Um ponto essencial é que **os raios μ dos nós internos nunca são alterados** durante a inserção. Como x é sempre enviado para o lado correto de cada bola, a propriedade de partição continua válida para todos os ancestrais. O único caso especial ocorre quando a descida termina em uma **folha** (um nó sem filhos e sem raio definido): nesse caso, a folha passa a ter μ = d(vp, x) e x é anexado como seu filho externo (pois d(vp, x) ≥ μ).

PSEUDOCÓDIGO DA INSERÇÃO

```
Função Inserir(Nó N, Ponto x):
    Se N for NULO:
        Retorne criar_folha(x)

    dist = d(N.ponto, x)

    // Caso especial: N é uma folha, ainda sem raio definido
    Se N.interno == NULO e N.externo == NULO:
        N.mu = dist
        N.externo = criar_folha(x)        // dist >= mu
        Retorne N

    Se dist < N.mu:
        N.interno = Inserir(N.interno, x)
    Senão:
        N.externo = Inserir(N.externo, x)
    Retorne N
```

Como a inserção não reorganiza a árvore, uma sequência longa de inserções pode desbalanceá-la, especialmente se os novos elementos se concentrarem em uma mesma região do espaço. Na prática, adota-se uma das seguintes estratégias: (i) reconstruir a árvore inteira (ou a subárvore mais desbalanceada) quando sua altura ultrapassar um limite como $2 \cdot \log_2 n$, de forma semelhante às *scapegoat trees*; ou (ii) acumular as inserções em um buffer e reconstruir periodicamente. Fu et al. (2000) estudam versões dinâmicas da VP-Tree que realizam esses rebalanceamentos de forma local.

Exemplo de inserção

Considere a inserção do ponto **X(7,5)** na VP-Tree construída no exemplo anterior (Figura 2). O processo decorre da seguinte forma:

1. **Visita à raiz (A):** calcula-se $d(A, X) = \sqrt{(7-5)^2 + (5-5)^2} =$ **2,00**.
    - Como $2{,}00 < \mu(A) = 4{,}00$, a descida continua pela subárvore **interna**.
2. **Visita ao nó B:** calcula-se $d(B, X) = \sqrt{(7-6)^2 + (5-6)^2} =$ **1,41**.
    - Como $1{,}41 < \mu(B) = 2{,}92$, a descida continua pela subárvore **interna**.
3. **Chegada à folha D:** D não possui filhos nem raio definido. Calcula-se $d(D, X) = \sqrt{(7-5)^2 + (5-8)^2} =$ **3,61**.
    - O raio da folha passa a ser $\mu(D) = 3{,}61$.
    - X é anexado como filho **externo** de D, pois $d(D, X) = \mu(D)$.
4. **Resultado:** apenas três distâncias foram calculadas e nenhum raio de nó interno foi modificado. X satisfaz todas as restrições dos seus ancestrais: está dentro da bola de A e dentro da bola de B.

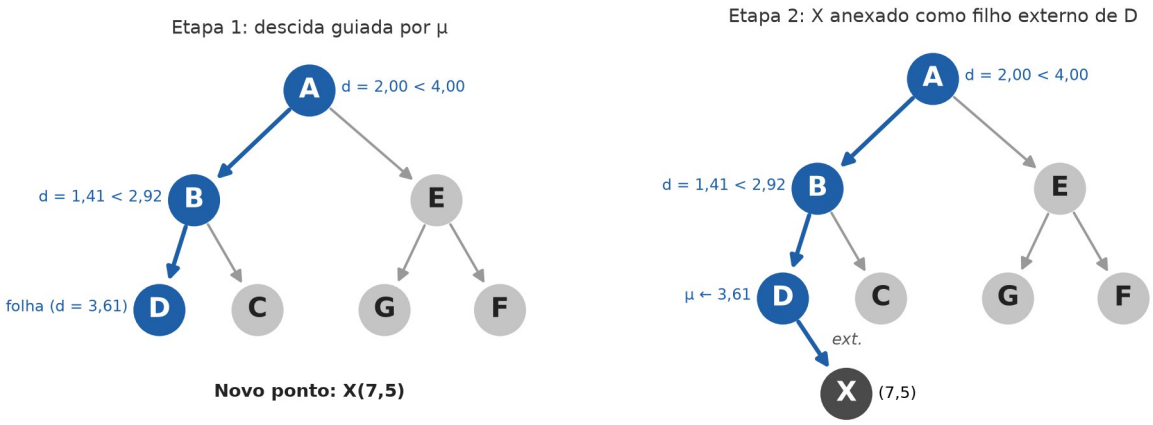


Figura 3

Figura 3: Inserção do ponto X(7,5). Na Etapa 1, a descida é guiada pela comparação entre a distância calculada e o raio μ de cada nó: em A (2,00 < 4,00) e em B (1,41 < 2,92) segue-se pela subárvore interna, chegando à folha D. Na Etapa 2, a folha D recebe o raio μ = 3,61 (a distância até X) e X é anexado como seu filho externo.

Remoção em uma VP-Tree

A remoção é a operação menos natural de uma VP-Tree. O motivo é que cada ponto armazenado em um nó interno não é apenas um dado, mas também o **centro de uma bola** que define a organização de toda a sua subárvore. Se o ponto de vantagem for simplesmente retirado, os raios μ deixam de ter significado e não existe uma forma simples de "promover" um filho para o lugar dele, como acontece em uma árvore binária de busca.

Por isso, a abordagem mais direta para remover um elemento x consiste em:

1. **Localizar** o nó que contém x, usando uma busca exata (descrita na próxima seção), que percorre um único caminho da raiz até o nó.
2. **Coletar** todos os elementos das subárvores interna e externa desse nó.
3. **Reconstruir** essa subárvore com a operação Construir, usando os elementos coletados, e colocá-la no lugar do nó removido.

A reconstrução é segura porque os elementos coletados são exatamente os mesmos que já estavam naquela subárvore e, portanto, continuam satisfazendo as restrições de distância de todos os ancestrais. Se o nó removido for uma folha, a operação se reduz a liberar o nó. Se for a raiz, a árvore inteira é reconstruída.

PSEUDOCÓDIGO DA REMOÇÃO

```
Função Remover(Nó N, Ponto x):
    Se N for NULO:
        Retorne NULO                        // x não está na árvore

    dist = d(N.ponto, x)

    Se dist == 0:                           // N contém x
        Lista S = Coletar(N.interno) + Coletar(N.externo)
        liberar_memoria(N)
        Retorne Construir(S)                // subárvore reconstruída

    Se dist < N.mu:
        N.interno = Remover(N.interno, x)
    Senão:
        N.externo = Remover(N.externo, x)
    Retorne N
```

Uma alternativa muito usada em aplicações com muitas remoções é a **remoção preguiçosa** (*lazy deletion*): o nó recebe uma marca de "removido", continua sendo usado como ponto de vantagem para guiar as buscas, mas deixa de ser retornado como resposta. Quando a fração de nós marcados ultrapassa um limite (por exemplo, 50%), a árvore é reconstruída apenas com os elementos válidos. Essa estratégia torna a remoção O(log n) em média, ao custo de uma reconstrução periódica.

Exemplo de remoção

Considere a remoção do ponto **B(6,6)** da VP-Tree resultante do exemplo de inserção, em que D possui X como filho externo. O processo decorre da seguinte forma:

1. **Localização de B:** a busca começa na raiz.
    - d(A, B) = 1,41 < μ(A) = 4,00, portanto a busca desce pela subárvore interna.
    - No nó seguinte, d(B, B) = 0: o nó que contém B foi encontrado.
2. **Coleta dos descendentes:** percorre-se a subárvore de B, obtendo a lista {D, X, C}.
3. **Reconstrução:** executa-se Construir({D, X, C}).
    - O ponto de vantagem é D. As distâncias são d(D, X) = 3,61 e d(D, C) = 4,47.
    - A mediana é (3,61 + 4,47) / 2 = **4,04**; X vai para a subárvore interna e C para a externa.
4. **Resultado:** a nova subárvore, com raiz D (μ = 4,04), ocupa o lugar de B como filho interno de A. A subárvore externa de A não foi tocada.

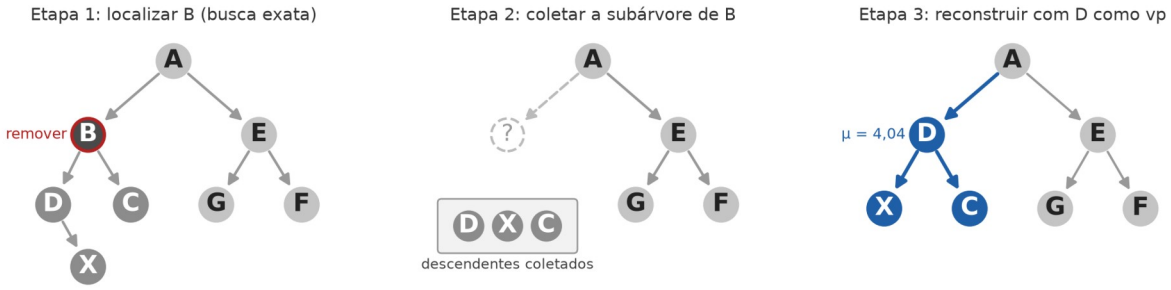


Figura 4

Figura 4: Remoção do ponto B. Na Etapa 1, B é localizado por uma busca exata que percorre apenas o caminho A → B. Na Etapa 2, o nó B é retirado e seus descendentes D, X e C são coletados. Na Etapa 3, esses elementos são usados para reconstruir a subárvore, agora com D como ponto de vantagem (μ = 4,04), X como filho interno e C como filho externo.

Busca em uma VP-Tree

A busca é a razão de existir da VP-Tree. Diferentemente das heaps, cujo objetivo é dar acesso rápido ao elemento de maior prioridade, a VP-Tree foi projetada justamente para responder consultas de **proximidade** de forma eficiente. Três tipos de busca são comuns:

- **Busca exata:** verificar se um elemento x está armazenado na árvore.
- **Busca por faixa** (*range search*): encontrar todos os elementos a uma distância menor ou igual a r de uma consulta q.
- **Busca do vizinho mais próximo** (*nearest neighbor*, NN) e sua generalização para os *k* vizinhos mais próximos (k-NN).

O princípio da poda pela desigualdade triangular

Todas as buscas da VP-Tree se baseiam no mesmo princípio. Suponha que estamos procurando elementos a uma distância menor que τ (tau) de uma consulta q e que, em um nó com ponto de vantagem vp e raio μ, já calculamos d(q, vp). Pela desigualdade triangular:

- Para qualquer elemento x da subárvore **interna** (d(vp, x) < μ), vale d(q, x) ≥ d(q, vp) − d(vp, x) > d(q, vp) − μ. Portanto, se **d(q, vp) − τ ≥ μ**, nenhum elemento interno pode estar a menos de τ de q, e a subárvore interna inteira pode ser **descartada**.
- Para qualquer elemento x da subárvore **externa** (d(vp, x) ≥ μ), vale d(q, x) ≥ d(vp, x) − d(q, vp) ≥ μ − d(q, vp). Portanto, se **d(q, vp) + τ < μ**, nenhum elemento externo pode estar a menos de τ de q, e a subárvore externa inteira pode ser **descartada**.

Em termos geométricos, a "bola de busca" de centro q e raio τ é comparada com a bola do nó: se ela estiver inteiramente de um dos lados da fronteira de raio μ, apenas aquele lado precisa ser explorado. Somente quando a bola de busca cruza a fronteira é que as duas subárvores precisam ser visitadas. A Figura 5 ilustra os dois casos de poda.

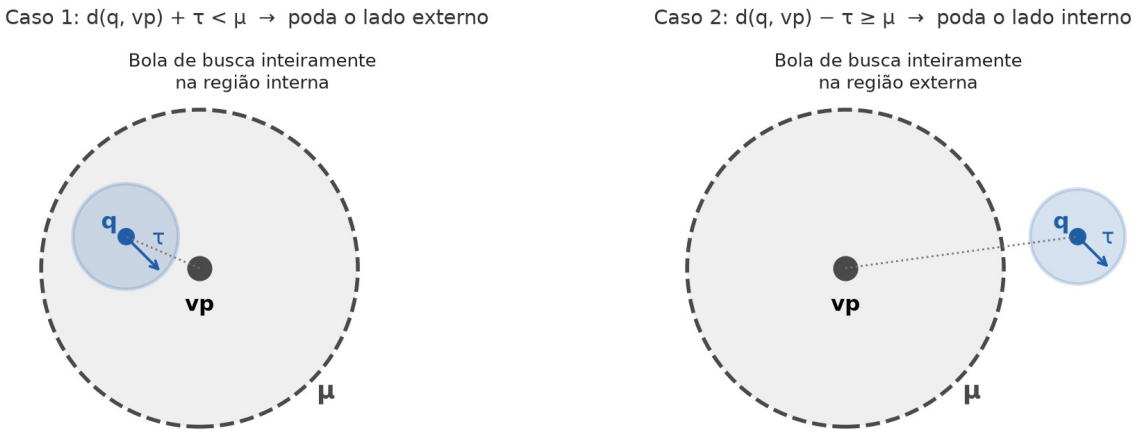


Figura 5

Figura 5: Princípio da poda pela desigualdade triangular. No Caso 1, a bola de busca (centro q, raio τ) está inteiramente dentro da bola do nó, pois d(q, vp) + τ < μ; nenhum elemento externo pode estar a menos de τ de q e a subárvore externa é descartada. No Caso 2, a bola de busca está inteiramente fora, pois d(q, vp) − τ ≥ μ, e a subárvore interna é descartada.

Busca exata

Para verificar se um elemento x pertence à árvore, basta aplicar o princípio acima com raio τ = 0. Nesse caso, as condições de visita se tornam mutuamente exclusivas: visita-se a subárvore interna somente se d(x, vp) < μ e a externa somente se d(x, vp) ≥ μ. A busca exata percorre, portanto, **um único caminho** da raiz até uma folha, exatamente como em uma árvore binária de busca, e é a mesma descida usada na inserção e na localização para remoção.

PSEUDOCÓDIGO DA BUSCA EXATA

```
Função BuscarExato(Nó N, Ponto x):
    Se N for NULO:
        Retorne FALSO
    dist = d(N.ponto, x)
    Se dist == 0:
        Retorne VERDADEIRO
    Se dist < N.mu:
        Retorne BuscarExato(N.interno, x)
    Senão:
        Retorne BuscarExato(N.externo, x)
```

Busca por faixa

Na busca por faixa, o raio de busca r é fixo. Em cada nó, o ponto de vantagem é incluído no resultado se estiver a uma distância de no máximo r da consulta, e cada subárvore só é visitada se a bola de busca puder intersectá-la.

PSEUDOCÓDIGO DA BUSCA POR FAIXA

```
Função BuscarFaixa(Nó N, Ponto q, Real r, Lista R):
    Se N for NULO:
        Retorne
    dist = d(q, N.ponto)
    Se dist <= r:
        Adicionar N.ponto em R
    Se dist - r < N.mu:                    // bola de busca toca o interior
        BuscarFaixa(N.interno, q, r, R)
    Se dist + r >= N.mu:                   // bola de busca toca o exterior
        BuscarFaixa(N.externo, q, r, R)
```

Busca do vizinho mais próximo

Na busca do vizinho mais próximo, o raio de busca τ não é conhecido de antemão. Ele começa infinito e vai **diminuindo** à medida que elementos mais próximos são encontrados: τ é sempre a distância até o melhor candidato atual. Para encontrar bons candidatos cedo e, assim, podar mais, a busca visita primeiro a subárvore do mesmo lado da fronteira em que a consulta se encontra. Para os *k* vizinhos mais próximos, mantém-se uma max-heap com os *k* melhores candidatos e τ passa a ser a distância do pior deles.

PSEUDOCÓDIGO DA BUSCA DO VIZINHO MAIS PRÓXIMO

```
// Inicialização: melhor = NULO, tau = +infinito
Função BuscarVizinho(Nó N, Ponto q):
    Se N for NULO:
        Retorne
    dist = d(q, N.ponto)
    Se dist < tau:                         // novo melhor candidato
        tau = dist
        melhor = N.ponto

    Se dist < N.mu:                        // q está dentro da bola
        Se dist - tau <  N.mu: BuscarVizinho(N.interno, q)
        Se dist + tau >= N.mu: BuscarVizinho(N.externo, q)
    Senão:                                 // q está fora da bola
        Se dist + tau >= N.mu: BuscarVizinho(N.externo, q)
        Se dist - tau <  N.mu: BuscarVizinho(N.interno, q)
```

Observe que as condições são verificadas **depois** da visita à primeira subárvore, de modo que o valor de τ usado na decisão sobre a segunda subárvore já reflete os candidatos encontrados na primeira.

Exemplo de busca

Considere a busca do vizinho mais próximo da consulta **Q(4,4)** na VP-Tree da Figura 2 (sem as alterações dos exemplos de inserção e remoção). O processo recursivo decorre da seguinte forma:

1. **Visita à raiz (A):** $d(Q, A) = \sqrt{2} =$ **1,41**. Como τ era infinito, A passa a ser o melhor candidato e $\tau = 1{,}41$.
    - Como $1{,}41 < \mu(A) = 4{,}00$, Q está dentro da bola de A e a subárvore interna é visitada primeiro.

2. **Visita ao nó B:** d(Q, B) = √8 = **2,83**, que não melhora τ.
    - Como 2,83 < μ(B) = 2,92, visita-se primeiro a subárvore interna de B.
3. **Visita ao nó D:** d(Q, D) = √17 = **4,12**, que não melhora τ. D é folha.
    - De volta a B, verifica-se a subárvore externa: 2,83 + 1,41 = 4,24 ≥ 2,92, portanto ela precisa ser visitada.
4. **Visita ao nó C:** d(Q, C) = **1,00** < τ. C passa a ser o melhor candidato e τ = 1,00.
5. **Poda na raiz:** de volta a A, verifica-se a subárvore externa: d(Q, A) + τ = 1,41 + 1,00 = 2,41 < μ(A) = 4,00.
    - A bola de busca está inteiramente dentro da bola de A; a subárvore externa inteira (E, G e F) é **descartada** sem nenhum cálculo de distância.

O resultado é o ponto **C(3,4)**, a uma distância de 1,00 de Q, obtido com apenas 4 cálculos de distância em vez dos 7 de uma busca exaustiva. Em bases com milhares ou milhões de elementos, a proporção de nós descartados é muito maior, pois cada poda elimina uma subárvore inteira.

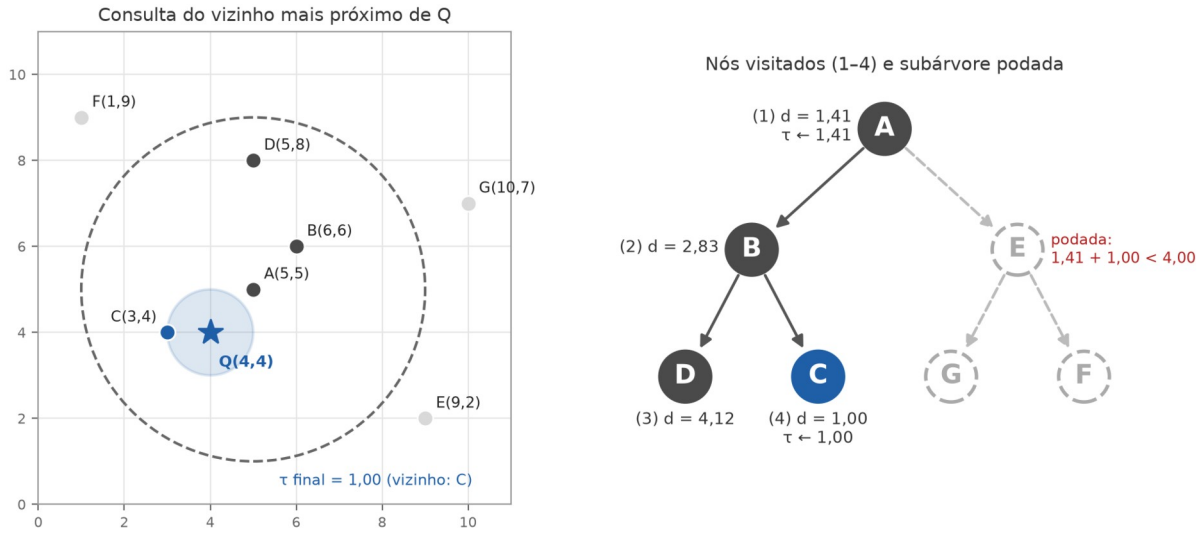


Figura 6

Figura 6: Busca do vizinho mais próximo de Q(4,4). À esquerda, a consulta no plano: a bola de busca final (raio τ = 1,00, em azul) está inteiramente dentro da bola da raiz A (μ = 4,00, tracejada), por isso nenhum ponto fora dela (E, G e F, em cinza claro) precisa ser examinado. À direita, a ordem de visita dos nós (1 a 4), as distâncias calculadas e a subárvore externa de A, podada porque 1,41 + 1,00 < 4,00.

Complexidade das Operações

A VP-Tree construída pela mediana é **perfeitamente balanceada**: cada nó divide seus elementos em duas metades, e a altura da árvore é $\lceil \log_2(n+1) \rceil$. Por isso, as operações que percorrem um único caminho (busca exata, inserção e localização para remoção) custam O(log n) em uma árvore recém-construída, mesmo no pior caso. O pior caso O(n) para essas operações só aparece depois de muitas inserções concentradas em uma mesma região, que podem degenerar a árvore em uma lista.

Na construção, cada nível da recursão calcula O(n) distâncias e encontra a mediana em tempo O(n) esperado (com o algoritmo *quickselect*), totalizando O(n log n). Já as buscas por similaridade (faixa e vizinho mais próximo) dependem da **dimensão intrínseca** dos dados e do raio de busca: quando a bola

de busca é pequena em relação aos raios μ, quase todas as decisões levam à poda de uma das subárvores e a busca visita O(log n) nós em média (YIANILOS, 1993). Em dimensões muito altas, porém, as distâncias tendem a se concentrar em torno de um mesmo valor, a bola de busca cruza quase todas as fronteiras e a busca se aproxima de O(n).

Operação	**Caso médio**	**Pior caso**
Construção	O(n log n)	$O(n^2)$ [1]
Busca exata	O(log n)	O(n) [2]
Inserção	O(log n)	O(n) [2]
Remoção (reconstrução da subárvore)	O(log n + m log m) [3]	O(n log n)
Remoção preguiçosa (lazy)	O(log n)	O(n) [2]
Vizinho mais próximo (k-NN)	O(log n) [4]	O(n)
Busca por faixa	O(log n + k) [4]	O(n)
Espaço	O(n)	O(n)

[1] Quando o *quickselect* escolhe pivôs ruins sistematicamente; com o algoritmo da mediana das medianas, a construção é O(n log n) no pior caso. [2] Apenas após inserções que desbalanceiam a árvore; em uma árvore recém-construída, a altura é $\log_2 n$. [3] m é o tamanho da subárvore do nó removido; em média, a maior parte dos nós está próxima às folhas e m é pequeno. [4] Valores esperados para dados de baixa dimensão intrínseca e raios de busca pequenos; k é o número de elementos retornados.

Quanto ao espaço, a VP-Tree armazena exatamente um elemento por nó, além de um número real e dois ponteiros, ocupando O(n) memória. Não há duplicação de elementos nem informação de balanceamento, o que a torna bastante econômica em comparação com estruturas que armazenam os dados apenas nas folhas.

Aplicações da VP-Tree

A VP-Tree é indicada para qualquer problema em que seja preciso encontrar, de forma repetida, os elementos mais parecidos com uma consulta, especialmente quando a função de distância é **cara** ou quando os dados **não possuem coordenadas**. Seu diferencial em relação à KD-Tree está justamente aí: ela exige apenas que a função de distância seja uma métrica, o que amplia muito o leque de aplicações.

Uma aplicação clássica é a **busca de imagens por conteúdo** (*content-based image retrieval*). Cada imagem é representada por um vetor de características ou por um *hash* perceptual, e as imagens semelhantes a uma consulta são aquelas cujos descritores estão próximos. Com *hashes* binários comparados pela distância de Hamming, a VP-Tree permite, por exemplo, detectar fotos duplicadas ou quase duplicadas em grandes acervos sem comparar cada par de imagens.

No **reconhecimento facial e biométrico**, redes neurais transformam cada rosto, impressão digital ou voz em um vetor numérico (*embedding*), e a identificação de uma pessoa consiste em encontrar o vetor cadastrado mais próximo do vetor capturado. A VP-Tree acelera essa busca de vizinho mais próximo em bases com muitos indivíduos cadastrados.

Em **aprendizado de máquina**, a VP-Tree é usada para acelerar classificadores e regressores do tipo k-NN e algoritmos que dependem dos vizinhos de cada ponto. Um exemplo conhecido é o algoritmo de visualização **t-SNE** na sua versão Barnes-Hut: van der Maaten (2014) utiliza uma VP-Tree para

encontrar os vizinhos mais próximos de cada ponto e, assim, construir uma matriz de similaridades esparsa, reduzindo o custo do algoritmo de $O(n^2)$ para $O(n \log n)$.

No **processamento de texto**, a distância de edição de Levenshtein entre palavras é uma métrica. Uma VP-Tree construída sobre um dicionário permite encontrar rapidamente as palavras a uma distância de edição pequena de uma palavra digitada incorretamente, o que é útil em **corretores ortográficos**, sugestões de busca e deduplicação de nomes em cadastros. De forma análoga, em **bioinformática** sequências de DNA ou de proteínas podem ser indexadas por distâncias de edição para encontrar sequências semelhantes.

Em **sistemas de informação geográfica**, a distância sobre a superfície da Terra (fórmula de haversine) também é uma métrica. A VP-Tree pode indexar diretamente pontos de interesse definidos por latitude e longitude para responder perguntas como "quais são os cinco hospitais mais próximos desta localização?", sem a necessidade de projetar as coordenadas em um plano.

Por fim, a VP-Tree é usada em **reconhecimento de áudio** (*audio fingerprinting*), em **sistemas de recomendação** que buscam usuários ou itens com perfis semelhantes e, de forma geral, como componente básico de outras estruturas métricas. Sua simplicidade também a torna um excelente exemplo didático de como uma propriedade matemática, a desigualdade triangular, pode ser transformada diretamente em uma regra de poda eficiente.

Comparação com Outras Estruturas Espaciais

A VP-Tree pertence à família das estruturas de **indexação espacial e métrica**, que compartilham o mesmo objetivo: responder consultas de proximidade sem comparar a consulta com todos os elementos da base. A diferença principal entre essas estruturas está no **tipo de espaço** que exigem, na forma como **particionam** os dados e na informação que cada nó precisa guardar.

Característica	KD-Tree	Ball Tree	VP-Tree	BK-Tree
Espaço exigido	Vetorial (coordenadas)	Vetorial (usa centroides)	Métrico geral (só distâncias)	Métrico com distâncias inteiras
Critério de partição	Hiperplano alinhado a um eixo	Bolas aninhadas que envolvem grupos de pontos	Bola de raio μ centrada no ponto de vantagem	Um filho para cada valor de distância
Informação por nó	Ponto, eixo de corte	Centro e raio da bola	Ponto e raio μ	Ponto e filhos indexados pela distância
Aridade	Binária	Binária	Binária	Multivia
Construção	O(n log n)	O(n log n)	O(n log n)	Incremental (inserções)
Alta dimensão	Degrada rapidamente	Degrada mais lentamente	Degrada mais lentamente	Depende do intervalo de distâncias
Uso típico	Pontos 2D/3D, geometria	k-NN em vetores	Similaridade em espaços métricos	Correção ortográfica

A comparação mais direta é com a **KD-Tree**, a estrutura espacial mais conhecida. Para pontos em duas ou três dimensões, a KD-Tree costuma ser mais rápida, pois a comparação de uma única coordenada é

mais barata que o cálculo de uma distância completa. Contudo, ela depende de coordenadas e de cortes alinhados aos eixos, que perdem eficiência quando o número de dimensões cresce. A VP-Tree usa a distância completa em cada decisão e, por isso, se adapta melhor a dados de dimensão moderada e funciona em espaços sem coordenadas.

A **Ball Tree** também particiona o espaço com bolas, mas cada nó representa uma bola que **envolve** um grupo de pontos, cujo centro geralmente é um centroide (um ponto que pode não pertencer à base). Isso exige operações vetoriais. A VP-Tree, ao contrário, usa como centro um elemento da própria base e particiona pelo raio mediano, o que a torna aplicável a qualquer métrica.

A **BK-Tree** (Burkhard-Keller) é outra estrutura métrica, voltada para distâncias **discretas**, como a distância de Levenshtein. Cada nó possui um filho para cada valor inteiro de distância, formando uma árvore multivia. Ela é muito usada em corretores ortográficos, mas não se aplica a distâncias contínuas e não garante balanceamento. Já variações da própria VP-Tree, como a **MVP-Tree** (BOZKAYA; ÖZSOYOGLU, 1997), usam vários pontos de vantagem por nó e vários raios de corte, aumentando a aridade da árvore e reduzindo o número de cálculos de distância.

Assim como a Skew Heap simplifica a Leftist Heap ao eliminar informações de balanceamento, a VP-Tree simplifica a indexação espacial ao eliminar a dependência de coordenadas: tudo o que ela precisa é de uma função de distância que respeite a desigualdade triangular.

Considerações Finais

A VP-Tree mostra que é possível indexar dados para buscas por similaridade usando **apenas uma função de distância**, sem nenhuma suposição sobre coordenadas ou sobre a natureza dos objetos. Toda a estrutura se apoia em uma única ideia: cada nó escolhe um ponto de vantagem e um raio μ, separando os elementos próximos dos distantes, e a desigualdade triangular garante que, durante a busca, regiões inteiras possam ser descartadas com segurança.

A construção pela mediana produz uma árvore balanceada em $O(n \log n)$. A busca exata e a inserção percorrem um único caminho, com custo $O(\log n)$ em uma árvore balanceada, e a remoção é resolvida localizando o nó e reconstruindo apenas a sua subárvore, ou de forma preguiçosa, com reconstruções periódicas. As buscas por faixa e de vizinho mais próximo são o principal ponto forte da estrutura, visitando apenas uma pequena fração dos nós quando os dados possuem baixa dimensão intrínseca.

A principal contribuição didática da VP-Tree é ilustrar como uma propriedade matemática simples se transforma em um algoritmo de poda eficiente, e como uma mesma estrutura pode servir a domínios tão distintos quanto imagens, textos, sequências genéticas e coordenadas geográficas. Suas limitações, como a degradação em dimensões muito altas e a dificuldade de manter o balanceamento sob muitas atualizações, motivaram estruturas derivadas, como a MVP-Tree e a M-Tree, mas a ideia central continua a mesma. O funcionamento das operações descritas neste relatório pode ser observado na implementação em C entregue junto a este documento.

REFERÊNCIAS

BENTLEY, Jon Louis. Multidimensional binary search trees used for associative searching. **Communications of the ACM**, v. 18, n. 9, p. 509–517, 1975. DOI: https://doi.org/10.1145/361002.361007.

BOZKAYA, Tolga; ÖZSOYOGLU, Meral. Distance-based indexing for high-dimensional metric spaces. In: ACM SIGMOD INTERNATIONAL CONFERENCE ON MANAGEMENT OF DATA, 1997, Tucson. **Proceedings** [...]. New York: ACM, 1997. p. 357–368.

CHÁVEZ, Edgar; NAVARRO, Gonzalo; BAEZA-YATES, Ricardo; MARROQUÍN, José Luis. Searching in metric spaces. **ACM Computing Surveys**, v. 33, n. 3, p. 273–321, 2001. DOI: https://doi.org/10.1145/502807.502808.

FU, Ada Wai-chee; CHAN, Polly Mei-shuen; CHEUNG, Yin-Ling; MOON, Yiu Sang. Dynamic vp-tree indexing for n-nearest neighbor search given pair-wise distances. **The VLDB Journal**, v. 9, n. 2, p. 154–173, 2000.

UHLMANN, Jeffrey K. Satisfying general proximity/similarity queries with metric trees. **Information Processing Letters**, v. 40, n. 4, p. 175–179, 1991. DOI: https://doi.org/10.1016/0020-0190(91)90074-R.

VAN DER MAATEN, Laurens. Accelerating t-SNE using tree-based algorithms. **Journal of Machine Learning Research**, v. 15, p. 3221–3245, 2014.

YIANILOS, Peter N. Data structures and algorithms for nearest neighbor search in general metric spaces. In: ACM-SIAM SYMPOSIUM ON DISCRETE ALGORITHMS, 4., 1993, Austin. **Proceedings** [...]. Philadelphia: SIAM, 1993. p. 311–321.